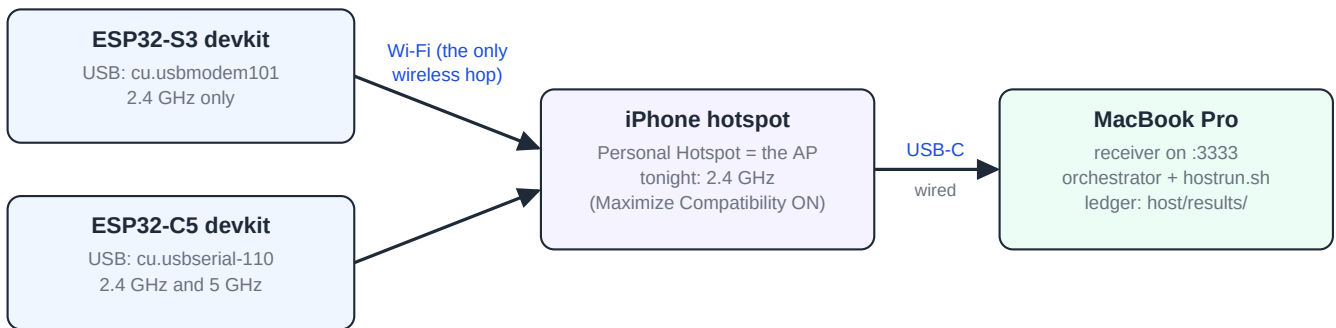


Overnight throughput run: ESP32-S3 vs ESP32-C5

Stage 1 runbook and test definition · hdemg-bench · written 2026-10-05

Tonight in one paragraph. Both ESP boards stream paced, counter-stamped dummy EMG frames over Wi-Fi to the iPhone hotspot, which passes them over a USB cable to the Mac. The Mac sweeps the offered load from 4 to 60 Mbit/s and records how much arrives, how much is lost and how much CPU each chip had left. This runs for the 2.4 GHz band only (both chips, 192 runs, about 4 hours). Tomorrow morning you flip one iPhone toggle and run the 5 GHz block (C5 only, 96 runs, about 2 hours). **No STM32 and no wiring are involved.** Your hands are needed for about 10 minutes tonight and one toggle in the morning.

1. The rig



The ESP-to-iPhone link is the only wireless hop; the iPhone-to-Mac link is a cable. If the Mac also joined the hotspot over Wi-Fi, every frame would cross the air twice on the same channel and every number would be roughly halved. Both boards also stay plugged into the Mac by USB, but that cable is used only for building and flashing firmware, never for data.

Program	Where it runs	What it does
hostrun.sh	Terminal A	The only thing allowed to build and flash. Reads small job files from <code>_agent/queue</code> , runs a fixed allow-list of commands, logs every build/flash to <code>_agent/audit.log</code> . You approve flashing once, for 12 hours.
overnight.sh	Terminal B	Keeps the Mac awake, checks hostrun.sh is alive, runs the sweep for one band, retries skipped points, restarts after a crash, renders the report at the end.
cli.bench run	started by overnight.sh	The orchestrator: decides which build each chip needs, asks hostrun for it, configures each run over UDP :3334, receives the stream on :3333, applies the validity gates, writes one ledger line per run.

2. What actually gets tested

Stage 1 of the test plan: each ESP generates its own frames at a commanded rate (the *synthetic source*), so the result isolates the Wi-Fi stack and the radio, with nothing wired in. It is the control arm that later, wired (STM32) results get subtracted from.

2.1 The matrix

Block	Chip	Band	Transport	Build (tune)	Runs
Tonight	ESP32-C5	2.4 GHz	TCP, UDP	baseline, tuned	4 cells × 24 = 96
Tonight	ESP32-S3	2.4 GHz	TCP, UDP	baseline, tuned	4 cells × 24 = 96
Morning	ESP32-C5	5 GHz	TCP, UDP	baseline, tuned	4 cells × 24 = 96

In total: **12 cells, 288 runs**. Each cell is swept over **8 offered loads: 4, 12, 20, 28, 36, 44, 52 and 60 Mbit/s**, and every load is measured **3 times** ($8 \times 3 = 24$ runs per cell). A run holds the load for 60 s and discards the first 3 s. The whole block runs once, then a second time, then a third, so repeats of the same point are hours apart: a phone that heats up shows as spread between repeats, not as a fake downward trend. The S3 has no 5 GHz cells because it has no 5 GHz radio. That asymmetry is part of the finding.

Each frame is 270 bytes: one 128-channel RAW16 sample (256 bytes of random payload) plus a 14-byte header carrying a sequence number and a source timestamp. The sequence number is how loss and reordering are counted.

2.2 What the two builds contain

Build	ESP32-S3	ESP32-C5
baseline (rung r0)	Stock ESP-IDF defaults. Wi-Fi power save off and TX power at maximum (set at runtime, identical on both chips).	Identical to the S3 baseline: this is the matched-configuration comparison.
tuned (top rung)	r2-core-split : CPU 240 MHz, -O2, logging off, <i>plus</i> Wi-Fi task on core 0 and the lwIP TCP/IP task on core 1.	r2-iram : CPU 240 MHz, -O2, logging off, <i>plus</i> Wi-Fi, Wi-Fi RX and lwIP hot paths placed in IRAM.

“Tuned” is provisional. It means the top of each chip's ladder as it exists today: three rungs per chip. The plan calls for climbing one lever at a time until three in a row gain less than 2 %, across about 13 common levers (Wi-Fi buffer counts, block-ack window, lwIP windows, core locking, PSRAM on the S3, HE settings on the C5, ...). None of that climbing has happened yet, and make effort still fails. So tonight's **baseline-vs-baseline** comparison is the solid result; tuned-vs-tuned is a first look.

2.3 What every run records

Measured by	Quantities
The Mac (receiver)	Goodput (Mbit/s) after the 3 s discard · UDP loss % from sequence gaps · reordered frames · latency p50 / p99 and jitter (relative: the ESP and Mac clocks are not synchronised, so this is spread around the fastest frame, not absolute delay)
The ESP (control plane)	CPU idle % per core (2 on the S3, 1 on the C5) · RSSI · PHY mode, channel and width (e.g. “HT ch6 bw20”; HE would mean Wi-Fi 6 is really in use) · free and minimum heap · frames sent / dropped · achieved vs commanded source rate
The harness	IDF version, firmware build and rung, git commit, rig ceiling used, Mac link (“usb” when the iPhone cable is up), timestamp, repeat number, gate results
Not recorded yet	TCP retransmits, Wi-Fi retry counters, absolute latency, ambient RF scan (needs sudo). The plan lists them; the harness does not collect them yet.

Every run becomes one JSON line in `host/results/runs.jsonl`. The ledger is append-only and is the only thing the report reads.

2.4 What the report works out

- **The knee of each cell:** the highest offered load whose *median* UDP loss over the 3 repeats stays under 0.1 %, and the goodput there (median ± median absolute deviation). TCP does not lose packets, so for TCP cells the number that matters is the goodput reached at the top load.
- **Maximum 128-channel sample rate** = $\text{goodput} \div (270 \times 8 \text{ bits})$, drawn against the plan's tiers: T0 5.0, T1 20.5, **T2 41.3 (the decision tier: 30 kS/s at the measured compression ratio 1.485)** and T3 61.4 Mbit/s.
- **SoC effect** (tonight): C5 at 2.4 GHz minus S3 at 2.4 GHz, same band, different silicon. **Band effect** (after the morning block): C5 at 5 GHz minus C5 at 2.4 GHz.
- **Baseline-to-tuned gap** per chip: how much performance the levers bought.
- **CPU idle % at the knee:** near 0 % means the CPU is the limit; above 20 % means the radio, the access point or protocol overhead is the limit.

Output: `host/results/report/report.md` plus figures (goodput vs load, loss vs load, baseline vs tuned, sample rate vs compression ratio), each in light and dark versions.

2.5 Validity gates: why some runs come back INVALID

A run that trips a gate is recorded as invalid and **retried in the next pass**. It never enters a median.

Gate	Fails the run when...
Band	the board reports a different band than the block expects (wrong iPhone toggle)
Toolchain	the board's IDF version differs from EXPECTED_IDF
Rig ceiling	the offered load exceeds the ceiling given for the band, or goodput beats it by more than 5 %
RSSI	signal strength moved more than 3 dB from that cell's first repeat (someone moved something). The most likely source of retries.
Delivery	no frames arrived, or the free heap fell below 20 kB
Reachability	the board did not answer even after being searched for again
Warnings only	source could not reach the commanded rate (cell labelled source-limited) · more than 1 % reordering · load above 80 % of the ceiling

2.6 Not tested tonight

Item	Why not
Stage 2: STM32H7 feeding the ESP over QSPI or SDIO	Not built yet. The ESP-side QSPI/SDIO sources are placeholders that refuse to compile, there is no STM32 firmware in the repo, and hostrun.sh has no STM32 build/flash actions. The wiring is a one-time manual job. The guide for the STM32 side is being written in a separate session.
Stage 0: stock ESP-IDF iperf reference	Not automated. The stage0 matrix as written runs the custom firmware, not iperf.
Real rig ceiling	Needs a second Wi-Fi device running iperf3 to the Mac. Tonight uses provisional ceilings of 60 (2.4 GHz) and 95 (5 GHz) Mbit/s.
ESP-IDF v6.0.2	Tonight uses the installed snapshot, v6.0-dev-2980-gab149384e1-dirty, on both chips. Every ledger line records it.
Tuning beyond rung r2	See the note in 2.2.

3. Step by step: tonight

All commands run from the repository root unless a step says otherwise:

```
cd "/Users/th3vik/Desktop/Research/NML work/ESP_COMPARISON_CODE/hdemg-bench"
```

Step 1. Bring in the fixes (once)

Your uncommitted edit to the S3 sdkconfig.defaults is the same cleanup the branch makes, so it is safe to discard.

```
git checkout -- firmware/esp32s3/sdkconfig.defaults
git merge --ff-only worktree-esp-overnight
```

Step 2. Physical setup

- **iPhone:** USB-C cable to the Mac. Settings > Personal Hotspot ON, **Maximize Compatibility ON** (= 2.4 GHz). Low Power Mode off, Auto-Lock Never, plugged in, left on the Personal Hotspot screen. Answer “Trust this computer” if asked.
- **Placement:** phone 1–2 m from both boards, clear line of sight, nothing moving near them overnight. Same spot every night.
- **Mac:** on AC power, **lid open** (closing the lid sleeps it). Bluetooth off on both Mac and iPhone for 2.4 GHz runs. Turning the Mac's Wi-Fi off is also recommended (a quieter band), and the Mac then reaches the internet through the phone.
- Both ESP boards plugged into the Mac by USB and powered.

Step 3. Terminal A: start the build/flash runner

```
FLASH_WINDOW_MIN=720 bash hostrun.sh
```

Leave this window open all night. Nothing happens in it until step 5 asks for a build.

Step 4. Pre-flight check (Terminal B)

```
(cd host && ../.venv/bin/python -m cli.bench discover)
```

You want to see iPhone USB address : 172.20.10.x. The boards may not show up yet if they are not running the bench firmware. That is fine: step 5 flashes them first. If the iPhone USB line says NOT UP, fix the cable or the hotspot before going on.

Step 5. Terminal B: start the 2.4 GHz block

```
CEILINGS=2.4=60,5=95 EXPECTED_IDF=v6.0-dev-2980-gab149384e1-dirty \
bash overnight.sh 2.4
```

Step 6. Approve flashing (about 2–3 minutes later)

When the first build finishes, **Terminal A** asks: type 'yes' to allow. Type **yes**. That one approval covers 12 hours of flashing; every flash is still written to the audit log.

Step 7. Watch the first few runs, then sleep

Healthy output in Terminal B looks like this: goodput close to the offered load at the low steps, and no INVALID flag:

```
found esp32c5 at 172.20.10.3 band=2.4 rung=r0-baseline ...
s1-esp32c5-2.4-synth-tcp-baseline off= 4.0 good= 3.98 loss= 0.00% rep1
s1-esp32c5-2.4-synth-tcp-baseline off= 12.0 good= 11.95 loss= 0.00% rep1
```

If the first two or three runs look like that, go to bed. Re-running the same command at any point is always safe: it picks up where it stopped, and finished points are never repeated.

What happens while you sleep

Approx. time	Event
0:00	Build + flash C5 baseline, then the C5 sweep (TCP then UDP, 8 loads each)
~0:20	Build + flash C5 tuned, sweep; then the S3 baseline and tuned, sweeps
~1:20	Repeat 2 starts on the build each chip already has, so only one reflash per chip
~2:40	Repeat 3
~4:00	Pass 2 (if needed) retries anything invalid or skipped; then the report is rendered and overnight.sh exits

8 flashes in total. The board that is not being measured is kept on the hotspot by a 30-second keepalive (the iPhone drops silent clients after 90 s). A board that drops off anyway rejoins on its own and is found again on the network.

4. Step by step: in the morning

- Check Terminal B ends with `=== all points done`. The full log is in `host/results/overnight-<date>-band2.4.log`.
- Open `host/results/report/report.md` and the figures next to it.
- Switch the iPhone to **Maximize Compatibility OFF** (= 5 GHz). The S3 drops off; that is expected, because it has no 5 GHz cells.
- Run the 5 GHz block. Terminal A's 12-hour approval is still valid if it was not restarted; otherwise type yes again.

```
CEILINGS=2.4=60,5=95 EXPECTED_IDF=v6.0-dev-2980-gab149384e1-dirty \  
bash overnight.sh 5
```

The report rendered at the end of that block covers both bands, including the band effect.

5. If something goes wrong

You see	Meaning	Do this
hostrun.sh is not running	Terminal A is not up	Do step 3, then step 5 again
iPhone USB address: NOT UP	No USB link to the phone	Re-plug, accept Trust, make sure Personal Hotspot is on
no bench devices answered	No board on the hotspot	Before the first flash: normal. After: check the band toggle (the S3 cannot join 5 GHz) and the SSID/password in <code>firmware/*/sdkconfig.local</code>
IDF ... != expected ...	EXPECTED_IDF does not match the firmware	Use the exact string the board reports (shown by discover)
hotspot on band 5, expected 2.4	Wrong iPhone toggle	Fix Maximize Compatibility, re-run
RSSI drifted ... dB	Something moved near the boards	Nothing; the next pass retries it
... the flash did not take	The board is not running the requested build	Look at the newest <code>_agent/out/*.log</code> for the build error
Terminal A waiting at 'type yes'	Approval window lapsed	Type yes
passes exhausted with points remaining	Some cells kept failing	Read the <code>abandoned_cells</code> reasons in the log, fix, run the same command again
A board never answers again	Hard hang (there is no switchable USB hub)	Press RESET on that board, run the same command again

6. Where things live

Path	Contents
host/results/runs.jsonl	The ledger: one JSON line per run. The raw data.
host/results/overnight-*.log	Everything Terminal B printed
host/results/report/	report.md and the figures
_agent/audit.log, _agent/out/	Every build and flash, with full output
host/rungs/s3, host/rungs/c5	The tuning ladders (what baseline and tuned contain)
host/matrices/stage1.yaml	The test matrix: loads, hold time, repeats